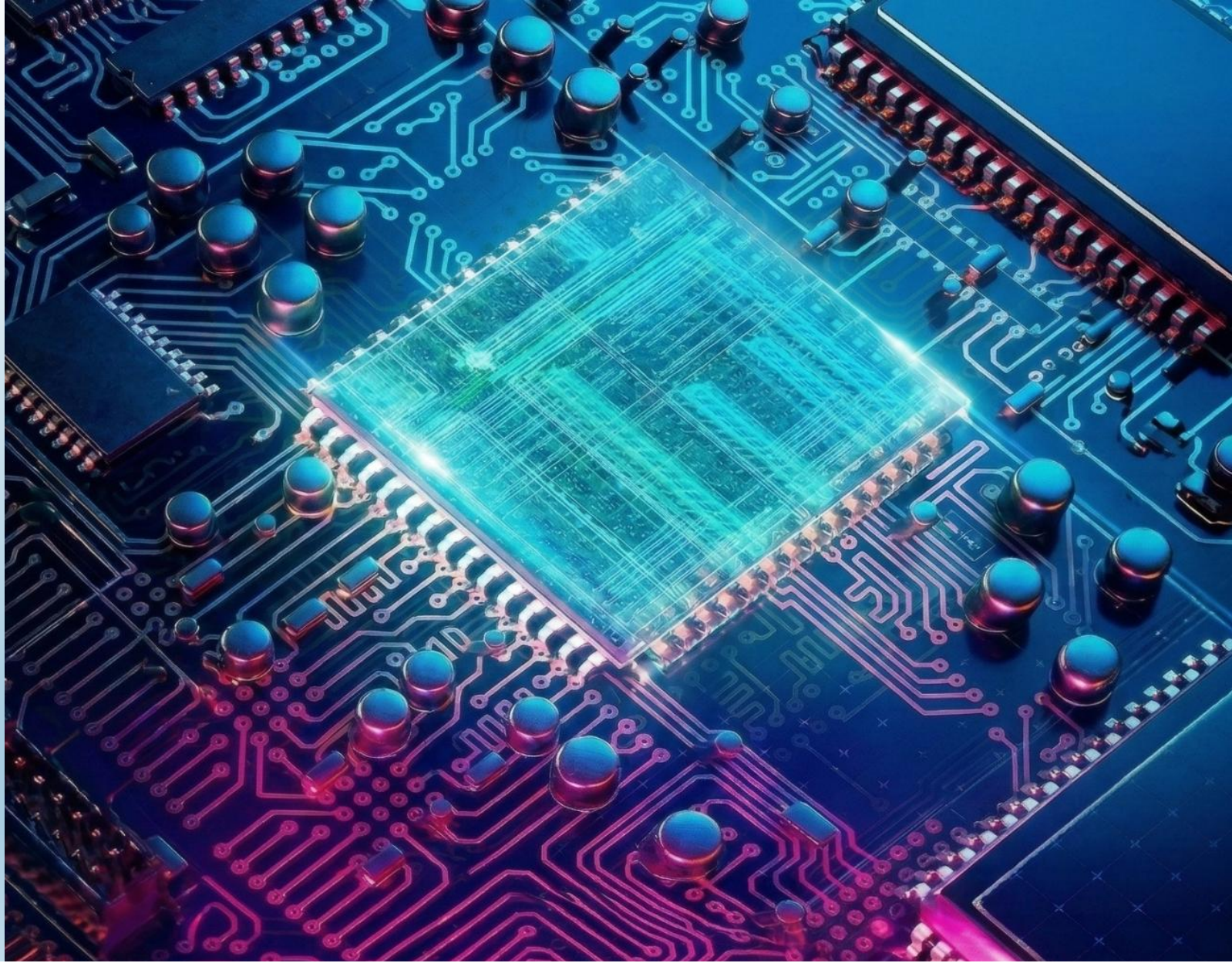




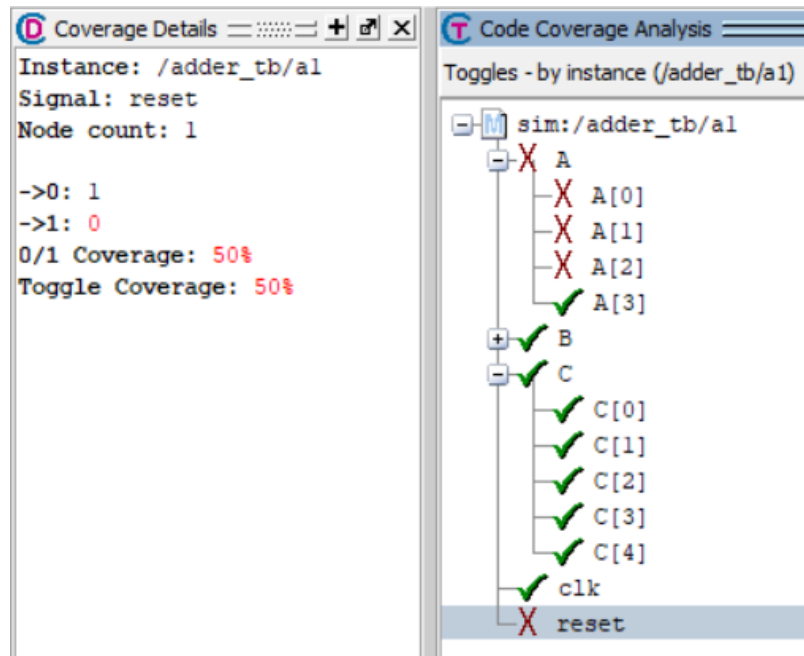
Digital Verification Assignment 1

Name: Mohamed Torki

Alexandria University, Faculty of Engineering
Electronics and Communications Department

Question 1:

Toggle Coverage for Class Testbench is only **50%** for **A[2:0]** and **reset** signals



We find out that **A[2:0]** perform from **0->1** only and **reset** performs **1->0** only

```
// ADDER_11
A = MAXNEG; B = MAXPOS;
check_result(-1);

// ADDER_12
assert_reset;
```

After adding these lines after **ADDER_10** The Toggle Coverage becomes **100%** for all signals



So, our Verification Plan Becomes

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ADDER_1	When the reset is asserted, the output C value must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_2	Verifying maximum negative value on A and maximum negative value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_3	Verifying maximum negative value on A and zero value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_4	Verifying maximum negative value on A and maximum positive value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_5	Verifying zero on A and maximum negative value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_6	Verifying zero on A and zero on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_7	Verifying zero on A and maximum positive value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_8	Verifying maximum positive value on A and maximum negative value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_9	Verifying maximum positive value on A and zero on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_10	Verifying maximum positive value on A and maximum positive value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_11	Verifying maximum positive value on A and maximum negative value on B	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ADDER_12	When the reset is asserted, the output C value must be low	Directed at the end of the simulation	-	A checker in the testbench to make sure the output is correct

New Toggle Coverage:

Toggle Coverage Report.txt

```

=====Toggle Details=====

Toggle Coverage for instance /adder_tb/a1 --

Node          1H->0L    0L->1H    "Coverage"
-----
A[0-3]         1           1      100.00
B[0-3]         1           1      100.00
C[4-0]         1           1      100.00
  clk          1           1      100.00
  reset        1           1      100.00

Total Node Count    =      15
Toggled Node Count  =      15
Untoggled Node Count =       0

Toggle Coverage     =    100.00% (30 of 30 bins)

```



Question 2:

Design issues:

```
# ** Error: priority_enc.v(11): (vlog-2110) Illegal reference to net "Y".
# ** Error: priority_enc.v(14): (vlog-2110) Illegal reference to net "Y".
# ** Error: priority_enc.v(15): (vlog-2110) Illegal reference to net "Y".
# ** Error: priority_enc.v(16): (vlog-2110) Illegal reference to net "Y".
# ** Error: priority_enc.v(17): (vlog-2110) Illegal reference to net "Y".
```

Y is called inside a sequential always block while missing adding **reg** keyword when declaring the signal

```
module priority_enc (
  input  clk,
  input  rst,
  input  [3:0] D,
  output [1:0] Y, //error
  output reg valid
);

always @(posedge clk) begin
  if (rst)
    Y <= 2'b0;
  else
    casex (D)
      4'b1000: Y <= 0;
      4'bX100: Y <= 1;
      4'bXX10: Y <= 2;
      4'bXXX1: Y <= 3;
    endcase
    valid <= (~|D)? 1'b0: 1'b1;
end
endmodule
```

Faulty Design

```
output reg [1:0] Y, //error
```

Correction



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
priority_enc 0	When the reset is asserted, the outputs Y and valid value must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
priority_enc 1	generating all possible input combinations for the input D	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

Testbench:

Assignments > A1 > Q2 > priority_enc_tb.SV

```

1  module priority_enc_tb ;
2
3  bit clk_tb, rst_tb;
4  logic [3:0] D_tb;
5  logic [1:0] Y_tb;
6  logic valid_tb;
7
8  logic valid_ref;
9  logic [1:0] Y_ref;
10
11 integer correct_count, error_count;
12
13 priority_enc DUT0(
14     .clk(clk_tb),
15     .rst(rst_tb),
16     .D(D_tb),
17     .Y(Y_tb),
18     .valid(valid_tb)
19 );
20
21 initial begin
22     clk_tb = 0;
23     forever
24         #1 clk_tb = ~clk_tb;
25 end

```

Declaring Signals,
Instantiating DUT and
Generating a clock

```

27 initial begin
28     correct_count = 0;
29     error_count = 0;
30     rst_tb = 0;
31     D_tb = 0;
32
33 //priority_enc 0
34 assert_reset();
35 @(negedge clk_tb);
36 assert_reset();
37
38 //priority_enc 1
39 repeat (17) begin
40     D_tb++;
41     @(negedge clk_tb);
42     {valid_ref, Y_ref} = expected_result(rst_tb, D_tb);
43     check_result(valid_ref, Y_ref);
44 end
45
46 $display("%t: At end of test error count is %0d and correct count = %0d",
47         $time, error_count, correct_count);
48 $stop;
49 end

```

Verifying reset functionality

Testing all possible inputs



```

51 task check_result(input logic valid_expected, input logic [2:0] Y_expected);
52     @(negedge clk_tb)
53     if(Y_expected != Y_tb | valid_expected != valid_tb)
54     begin
55         $display("At Time %t: D_tb = %0d | rst_tb = %0d | Y_tb = %0d |
56         | Y_expected = %0d | Valid_tb = %0d | Valid_expected = %0d",
57         $time,D_tb,rst_tb,Y_tb,Y_expected,valid_tb,valid_expected);
58         error_count++;
59     end
60     else
61         correct_count++;
62 endtask
63
64 task assert_reset;
65     rst_tb = 1;
66     check_result(0,0);
67     rst_tb = 0;
68 endtask
69

```

Assert Reset and Check result tasks

```

70 function logic [2:0] expected_result (input logic rst_fn, input logic [3:0] D_fn);
71     if (rst_fn)
72         expected_result = {1'b0, 2'b00};
73
74     else if (D_fn[0])
75         expected_result = {1'b1, 2'b11};
76
77     else if (D_fn[1])
78         expected_result = {1'b1, 2'b10};
79
80     else if (D_fn[2])
81         expected_result = {1'b1, 2'b01};
82
83     else if (D_fn[3])
84         expected_result = {1'b1, 2'b00};
85
86     else
87         expected_result = {1'b0, 2'bxx};
88 endfunction
89
90 endmodule

```

A function to generate Reference Signals



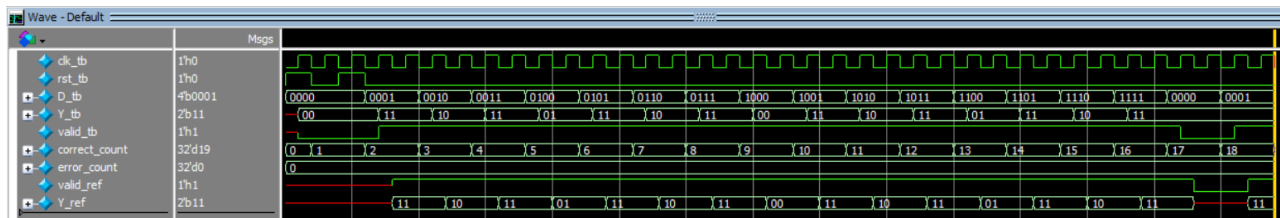
Do File:

```

Assignments > A1 > Q2 > ≡ run.do
1  vlib work
2  vlog priority_enc.v priority_enc_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.priority_enc_tb -cover
4  add wave *
5  coverage save priority_enc_tb.ucdb -onexit
6  run -all

```

Waveform:



Coverage Report:

```

Statement Coverage:
Enabled Coverage      Bins   Hits   Misses Coverage
-----
Statements            7      7      0 100.00%

=====Statement Details=====
Statement Coverage for instance /priority_enc_tb/DUT0 --
Line  Item          Count   Source
----  -
File priority_enc.v
1      module priority_enc (
2      input clk,
3      input rst,
4      input [3:0] D,
5      output reg [1:0] Y, //error
6      output reg valid
7      );
8
9      1              37  always @(posedge clk) begin
10     if (rst)
11     Y <= 2'b0;
12     else
13     casex (D)
14     1              2      4'b1000: Y <= 0;
15     1              4      4'b0100: Y <= 1;
16     1              8      4'b0010: Y <= 2;
17     1              18     4'b0001: Y <= 3;
18     endcase
19     1              37  valid <= (~D)? 1'b0: 1'b1;

```

Statement Coverage

```

Toggle Coverage:
Enabled Coverage      Bins   Hits   Misses Coverage
-----
Toggles              18     18      0 100.00%

=====Toggle Details=====
Toggle coverage for instance /priority_enc_tb/DUT0 --
Node  1H->0L  0L->1H  "Coverage"
-----
D[0-3] 1        1    100.00
Y[1-0] 1        1    100.00
clk     1        1    100.00
rst     1        1    100.00
valid   1        1    100.00

Total Node Count = 9
Toggled Node Count = 9
Untoggled Node Count = 0

Toggle Coverage = 100.00% (18 of 18 bins)

```

Toggle Coverage

```

Branch Coverage:
Enabled Coverage      Bins   Hits   Misses Coverage
-----
Branches              7      7      0 100.00%

=====Branch Details=====
Branch Coverage for instance /priority_enc_tb/DUT0
Line  Item          Count   Source
----  -
File priority_enc.v
-----IF Branch-----
10     1              37  Count coming in to IF
10     1              2      if (rst)
12     1              35  else
Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----
13     1              35  Count coming in to CASE
14     1              2      4'b1000: Y <= 0;
15     1              4      4'b0100: Y <= 1;
16     1              8      4'b0010: Y <= 2;
17     1              18     4'b0001: Y <= 3;
3      All False Count
Branch totals: 5 hits of 5 branches = 100.00%

```

Branch Coverage



Question 3:

Design issues:

```
# ** Error: C:/questasim64_2021.1/win64/vlog failed.
```

Module name (`ALU_4_bit`) is different from the file name (`ALU.v`)

```
module ALU_4_bit (
    input  clk,
    input  reset,
    input  [1:0] Opcode,    // The opcode
    input  signed [3:0] A,  // Input data A in 2's complement
    input  signed [3:0] B,  // Input data B in 2's complement

    output reg signed [4:0] C // ALU output in 2's complement

);
```

Faulty Design

```
1  module ALU (
2      input  clk,
3      input  reset,
4      input  [1:0] Opcode,    // The opcode
5      input  signed [3:0] A,  // Input data A in 2's complement
6      input  signed [3:0] B,  // Input data B in 2's complement
7
8      output reg signed [4:0] C // ALU output in 2's complement
9
10     );
```

Correction

Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALU_0	When the reset is asserted, the output C value must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
ALU_1	Verifying all possible corner values for A and B with Opcode 2'b00 (Addition)	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_2	Verifying all possible corner values for A and B with Opcode 2'b01 (Subtraction)	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_3	Verifying all possible values for A with Opcode 2'b10 (Bitwise NOT)	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_4	Verifying all possible values for A with Opcode 2'b11 (reduction OR)	Directed during the simulation	-	A checker in the testbench to make sure the output is correct
ALU_5	Verifying all possible values for the Opcode with A&B kept constant of values 7, -2	Directed during the simulation	-	A checker in the testbench to make sure the output is correct



Testbench:

```

Assignments > A1 > Q3 > ALU_tb.sv
1  module ALU_tb;
2
3  bit clk_tb, reset_tb;
4  logic [1:0] Opcode_tb;
5  logic signed [3:0] A_tb, B_tb;
6
7  logic signed [4:0] C_tb;
8
9  integer correct_count, error_count;
10 localparam [4:0] MAXPOS = 7, MAXNEG = -8, ZERO = 0;
11
12
13 ALU DUT0(
14     .clk(clk_tb),
15     .reset(reset_tb),
16     .Opcode(Opcode_tb),
17     .A(A_tb),
18     .B(B_tb),
19
20     .C(C_tb)
21 );
22
23 initial begin
24     clk_tb = 0;
25     forever
26     |   #1 clk_tb = ~clk_tb;
27 end

```

Declaring Signals,
Instantiating DUT and
Generating a clock

```

Assignments > A1 > Q3 > ALU_tb.sv

```

```

29  initial begin
30      correct_count = 0;
31      error_count = 0;
32      reset_tb = 0;
33      Opcode_tb = 0;
34      A_tb = 0;
35      B_tb = 0;
36
37      //ALU_0
38      assert_reset();
39      @(negedge clk_tb);
40      assert_reset();
41

```

Testing Reset functionality



Testing corner cases on Adding

```

Assignments > A1 > Q3 > ALU_tb.sv
42 //ALU_1
43 Opcode_tb =2'b00;
44 A_tb = MAXPOS; B_tb = MAXPOS;
45 check_result(14);
46
47 A_tb = MAXPOS; B_tb = MAXNEG;
48 check_result(-1);
49
50 A_tb = MAXPOS; B_tb = ZERO;
51 check_result(MAXPOS);
52
53 A_tb = MAXNEG; B_tb = MAXPOS;
54 check_result(-1);
55
56 A_tb = ZERO; B_tb = MAXPOS;
57 check_result(MAXPOS);
58
59 A_tb = ZERO; B_tb = MAXNEG;
60 check_result(MAXNEG);
61
62 A_tb = ZERO; B_tb = ZERO;
63 check_result(0);
64
65 A_tb = MAXNEG; B_tb = MAXNEG;
66 check_result(-16);
67
68 A_tb = MAXNEG; B_tb = ZERO;
69 check_result(MAXNEG);

```

Testing corner cases on Subtracting

```

Assignments > A1 > Q3 > ALU_tb.sv
71 //ALU_2
72 Opcode_tb =2'b01;
73 A_tb = MAXPOS; B_tb = MAXPOS;
74 check_result(ZERO);
75
76 A_tb = MAXPOS; B_tb = MAXNEG;
77 check_result(15);
78
79 A_tb = ZERO; B_tb = MAXPOS;
80 check_result(-7);
81
82 A_tb = ZERO; B_tb = MAXNEG;
83 check_result(8);
84
85 A_tb = MAXNEG; B_tb = MAXPOS;
86 check_result(-15);
87
88 A_tb = MAXPOS; B_tb = ZERO;
89 check_result(MAXPOS);
90
91 A_tb = MAXNEG; B_tb = MAXNEG;
92 check_result(ZERO);
93
94 A_tb = ZERO; B_tb = ZERO;
95 check_result(0);
96
97 A_tb = MAXNEG; B_tb = ZERO;
98 check_result(MAXNEG);

```

Testing All possible cases For Bitwise NOT for A and Reduction OR For B

```

Assignments > A1 > Q3 > ALU_tb.sv
101 //ALU_3
102 Opcode_tb =2'b10;
103 A_tb = 4'b0000;
104
105 repeat (17) begin
106     check_result($signed(~A_tb));
107     A_tb++;
108 end
109
110 //ALU_4
111 Opcode_tb =2'b11;
112 B_tb = 4'b0000;
113
114 repeat (17) begin
115     if(B_tb == 4'b0000)
116         check_result(0);
117     else
118         check_result(1);
119     B_tb++;
120 end
121

```



Assignments > A1 > Q3 > ALU_tb.sv

```

122 //ALU_5
123 A_tb = 4'b0111; B_tb = 4'b1110;
124 Opcode_tb = 2'b00;
125
126 repeat (5) begin
127     if(Opcode_tb == 2'b00)
128         check_result(A_tb + B_tb);
129
130     else if(Opcode_tb == 2'b01)
131         check_result(A_tb - B_tb);
132
133     else if(Opcode_tb == 2'b10)
134         check_result(5'b11000);
135
136     else if(Opcode_tb == 2'b11)
137         check_result(1);
138     Opcode_tb++;
139 end
140
141 Opcode_tb = 2'bxx;
142 check_result(0);
143
144 $display("%t: At end of test error count is %0d and correct count = %0d",
145         $time, error_count, correct_count);
146 $stop;
147 end
148

```

Testing all Opcodes
with two constant
values for A and B

Assignments > A1 > Q3 > ALU_tb.sv

```

150 task check_result (input signed [4:0] expected_result);
151     @(negedge clk_tb);
152     if ($signed(expected_result) != C_tb) begin
153         error_count = error_count + 1;
154         $display("%t: Error: For A_tb = %0d | B_tb = %0d | Opcode_tb = %0d |
155                 C_tb should equal %0d but is %0d",
156                 $time, A_tb, B_tb, Opcode_tb, expected_result, C_tb);
157     end
158     else
159         correct_count = correct_count + 1;
160 endtask
161
162
163 task assert_reset;
164     reset_tb = 1;
165     check_result(0);
166     reset_tb = 0;
167 endtask
168
169
170 endmodule

```

Check result and
assert reset tasks



Coverage Report:

```

Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              7       7       0    100.00%

=====Branch Details=====

Branch Coverage for instance /ALU_tb/DUT0

  Line      Item          Count    Source
  ----      -
  File ALU.v
  -----CASE Branch-----
  21          1          59    Count coming in to CASE
  22          1          12    Add:      Alu_out = A + B;
  23          1          10    Sub:      Alu_out = A - B;
  24          1          18    Not_A:    Alu_out = ~A;
  25          1          18    ReductionOR_B: Alu_out = |B;
  26          1           1    default: Alu_out = 5'b0;
Branch totals: 5 hits of 5 branches = 100.00%

-----IF Branch-----
  32          1          50    Count coming in to IF
  32          1           4    if (reset)
  34          1          46    else
Branch totals: 2 hits of 2 branches = 100.00%

```

Branch Coverage

```

Statement Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Statements            9       9       0    100.00%

=====Statement Details=====

Statement Coverage for instance /ALU_tb/DUT0 --

  Line      Item          Count    Source
  ----      -
  File ALU.v
  1          1          module ALU (
  2          2          input  clk,
  3          3          input  reset,

```

Statement Coverage

```

Toggle Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Toggles             44      44       0    100.00%

=====Toggle Details=====

Toggle Coverage for instance /ALU_tb/DUT0 --

  Node      1H->0L    0L->1H    "Coverage"
  ----
  A[0-3]      1         1    100.00
  Alu_out[4-0] 1         1    100.00
  B[0-3]      1         1    100.00
  C[4-0]      1         1    100.00
  Opcode[0-1] 1         1    100.00
  clk         1         1    100.00
  reset       1         1    100.00

Total Node Count = 22
Toggled Node Count = 22
Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

```

Toggle Coverage

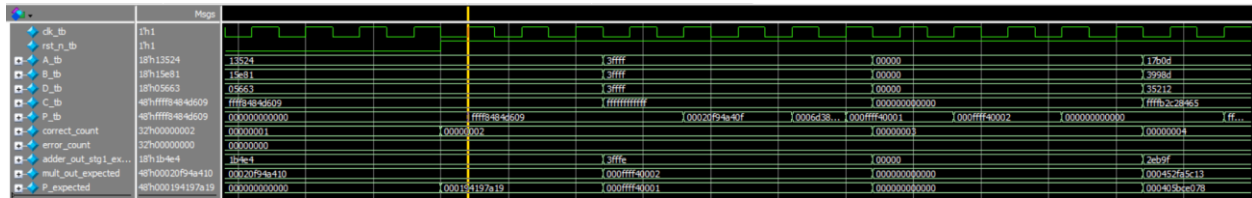


Question 4:

Design issues:

Here we find that P takes a value straight after the positive clock edge after de-asserting reset which is wrong compared to the designed provided.

After reset is asserted, P should be low at least for one clock cycle.



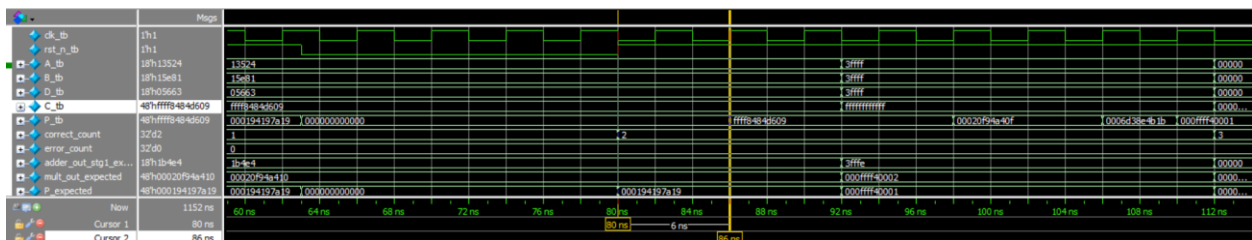
The Problem is that **C_reg** is not cleared when receiving a reset assertion

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // reset
        A_reg_stg1 <= 0;
        A_reg_stg2 <= 0;
        B_reg <= 0;
        D_reg <= 0;
        adder_out_stg1 <= 0;
        mult_out <= 0;
        P <= 0;
    end
```

Faulty Design

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // reset
        A_reg_stg1 <= 0;
        A_reg_stg2 <= 0;
        B_reg <= 0;
        D_reg <= 0;
        C_reg <= 0; // correction
        adder_out_stg1 <= 0;
        mult_out <= 0;
        P <= 0;
    end
```

Correction



Waveform after correcting



Verification Plan:

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
DSP_0	When the reset is asserted, the output P value must be low	Directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
DSP_1	Verifying some corner and random values for A, B, C and D (Addition Mode)	Directed during the simulation	-	A checker in the testbench to make sure the output is correct

Testbench:

```

Assignments > A1 > Q4 > DSP_tb.sv
1  module DSP_tb();
2  bit clk_tb, rst_n_tb;
3  logic [17:0] A_tb, B_tb, D_tb;
4  logic [47:0] C_tb;
5  logic [47:0] P_tb;
6
7  localparam OPERATION_tb = "ADD";
8  integer correct_count, error_count;
9  localparam MAXPOS_1 = 18'h3ffff, MAXPOS_2 = 48'hffffffffffff, ZERO = 18'h0000;
10
11 DSP #(.OPERATION(OPERATION_tb)) DUT0(
12     .A(A_tb),
13     .B(B_tb),
14     .C(C_tb),
15     .D(D_tb),
16     .clk(clk_tb),
17     .rst_n(rst_n_tb),
18     .P(P_tb)
19 );
20
21 initial begin
22     clk_tb = 0;
23     forever
24         #2 clk_tb = ~clk_tb;
25 end

```

**Declaring Signals,
Instantiating DUT and
Generating a clock**

```

Assignments > A1 > Q4 > DSP_tb.sv
28 //reference model
29 logic [17:0] adder_out_stg1_expected;
30 logic [47:0] mult_out_expected;
31 logic [47:0] P_expected;
32 always@(*) begin
33     if(!rst_n_tb)
34         P_expected = 0;
35     else begin
36         adder_out_stg1_expected = D_tb + B_tb ;
37         mult_out_expected = A_tb * adder_out_stg1_expected;
38         P_expected = C_tb + mult_out_expected;
39     end
40 end
41

```

Reference Model



```

Assignments > A1 > Q4 > DSP_tb.sv
42 initial begin
43     A_tb = 0; B_tb = 0; C_tb = 0; D_tb = 0; rst_n_tb = 1; correct_count = 0; error_count = 0;
44     repeat(5)@(negedge clk_tb);
45
46     //DSP_0
47     assert_reset();
48     A_tb = $random; B_tb = $random; C_tb = $random; D_tb = $random;
49     repeat(5)@(negedge clk_tb);
50     #3; //Checking Asynchronous property
51     assert_reset();
52
53     //DSP_1
54     repeat(3)@(negedge clk_tb);
55     A_tb = MAXPOS_1; B_tb = MAXPOS_1; C_tb = MAXPOS_2; D_tb = MAXPOS_1;
56     check_result();
57
58     A_tb = ZERO; B_tb = ZERO; C_tb = ZERO; D_tb = ZERO;
59     check_result();
60
61     for(int i = 0; i < 51; i++)begin
62         A_tb = $random; B_tb = $random; C_tb = $random; D_tb = $random;
63         check_result();
64     end
65
66     $display("%t: At end of test error count is %0d and correct count = %0d",
67             $time, error_count, correct_count);
68     $stop;
69 end

```

Testing Reset Assertion
Functionality with
some random values
and checking the
Asynchronous property

Testing Corner Cases
and testing several
random inputs using
a for loop

```

Assignments > A1 > Q4 > DSP_tb.sv
71 task check_result;
72
73     repeat(5)@(negedge clk_tb);
74     if(P_expected != P_tb)
75     begin
76         error_count ++;
77         $display("%t: Error: For A_tb = %0d | B_tb = %0d | C_tb = %0d | D_tb = %0d |
78             P_tb should equal %0d but is %0d",
79             $time, A_tb, B_tb, C_tb, D_tb, P_expected, P_tb);
80     end
81     else
82         correct_count++;
83 endtask
84
85 task assert_reset;
86     rst_n_tb = 0;
87     check_result();
88     rst_n_tb = 1;
89 endtask
90 endmodule

```

Check result and
assert reset tasks

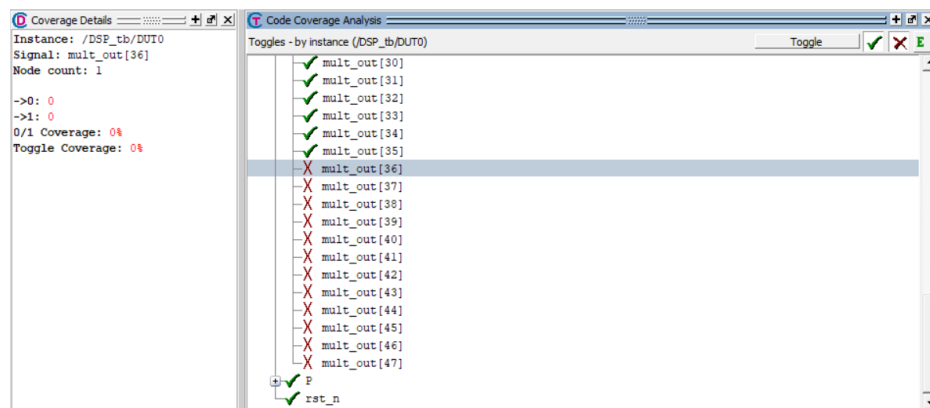


When checking the analysis window we find `mult_out[47:36]` bits aren't covered but why?

This comes from the fact that the multiplier's two operands are of 18-bit width.

It's known that if the width of a multiplier operands are $N1$ and $N2$ then result would be of width $\leq N1 + N2$ so in our case the maximum length is 36 Bits.

So here it's meaningless to keep track of `mult_out[47:36]` bits because they will never be stimulated therefore we are going to exclude them in the coverage report.



Do File:

```

Assignments > A1 > Q4 > run.do
1  vlib work
2  vlog DSP.v DSP_tb.sv +cover -covercells
3  vsim -voptargs=+acc work.DSP_tb -cover
4  add wave *
5  coverage save DSP_tb.ucdb -onexit
6  run -all
7  coverage exclude -du DSP -togglenode {mult_out[36]}
8  coverage exclude -du DSP -togglenode {mult_out[37]}
9  coverage exclude -du DSP -togglenode {mult_out[38]}
10 coverage exclude -du DSP -togglenode {mult_out[39]}
11 coverage exclude -du DSP -togglenode {mult_out[40]}
12 coverage exclude -du DSP -togglenode {mult_out[41]}
13 coverage exclude -du DSP -togglenode {mult_out[42]}
14 coverage exclude -du DSP -togglenode {mult_out[43]}
15 coverage exclude -du DSP -togglenode {mult_out[44]}
16 coverage exclude -du DSP -togglenode {mult_out[45]}
17 coverage exclude -du DSP -togglenode {mult_out[46]}
18 coverage exclude -du DSP -togglenode {mult_out[47]}

```

1152: At end of test error count is 0 and correct count = 55



Waveform:



Coverage Report:

Toggle Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	-----	-----	-----	-----
Toggles	688	688	0	100.00%
=====Toggle Details=====				
Toggle Coverage for instance /DSP_tb/DUT0 --				
	Node	1H->0L	0L->1H	"Coverage"
	A[0-17]	1	1	100.00
	A_reg_stg1[17-0]	1	1	100.00
	A_reg_stg2[17-0]	1	1	100.00
	B[0-17]	1	1	100.00
	B_reg[17-0]	1	1	100.00
	C[0-17]	1	1	100.00
	C_reg[17-0]	1	1	100.00
	D[0-17]	1	1	100.00
	D_reg[17-0]	1	1	100.00
	P[47-0]	1	1	100.00
	adder_out_stg1[17-0]	1	1	100.00
	adder_out_stg2[17-0]	1	1	100.00
	clk	1	1	100.00
	mult_out[35-0]	1	1	100.00
	rst_n	1	1	100.00
Total Node Count	=	344		
Toggled Node Count	=	344		
Untoggled Node Count	=	0		
Toggle Coverage	=	100.00% (688 of 688 bins)		

Toggle Coverage

Statement Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	-----	-----	-----	-----
Statements	18	18	0	100.00%
=====Statement Details=====				
Statement Coverage for instance /DSP_tb/DUT0 --				
Line	Item	Count	Source	
----	----	-----	-----	
File DSP.v				
1	module DSP(A, B, C, D, clk, rst_n, P);			
2	parameter OPERATION = "ADD";			
3	input [17:0] A, B, D;			
4	input [47:0] C;			
5	input clk, rst_n;			
6	output reg [47:0] P;			

Statement Coverage

Branch Coverage:				
Enabled Coverage	Bins	Hits	Misses	Coverage
-----	-----	-----	-----	-----
Branches	2	2	0	100.00%
=====Branch Details=====				
Branch Coverage for instance /DSP_tb/DUT0				
Line	Item	Count	Source	
----	----	-----	-----	
File DSP.v				
-----IF Branch-----				
12		284	Count coming in to IF	
12	1	4	if (rst_n) begin	
23	1	280	else begin	
Branch totals: 2 hits of 2 branches = 100.00%				

Branch Coverage

